

BlockBase – Day 5

Mentored by FEC

Some more Solidity

Arrays, Strings

▶ [Arrays and structs](#)

Arrays can have a compile-time fixed size or a dynamic size.

```
pragma solidity ^0.8.10;

contract Array {

    // Declare a string variable which is public
    string public greet = "Hello World!";
    // Several ways to initialize an array
    // Arrays initialized here are considered state variables that get stored on
the blockchain
    // These are called storage variables
    uint[] public arr;
    uint[] public arr2 = [1, 2, 3];
    // Fixed sized array, all elements initialize to 0
    uint[10] public myFixedSizeArr;
    /*
        Name of the function is get
        It gets the value of element stored in an array's index
    */
    function get(uint i) public view returns (uint) {
        return arr[i];
    }
}
```



```
/*  
Solidity can return the entire array.  
This function gets called with and returns a uint[] memory.  
memory - the value is stored only in memory and not on the blockchain it  
only exists during the time the function is being executed.
```

Memory variables and Storage variables can be thought of as similar to RAM vs Hard Disk.

Memory variables exist temporarily during function execution, whereas Storage variables are persistent across function calls for the lifetime of the contract. Here the array is only needed for the duration while the function executes and thus is declared as a memory variable

```
*/  
function getArr(uint[] memory _arr) public view returns (uint[] memory) {  
    return _arr;  
}
```

```
/*  
This function returns string memory.  
The reason memory keyword is added is because string internally works as  
an array  
Here the string is only needed while the function executes.
```

```
*/  
function foo() public returns (string memory) {  
    return "C";  
}
```

```
function doStuff(uint i) public {  
    // Append to array  
    // This will increase the array length by 1.  
    arr.push(i);  
    // Remove Last element from array  
    // This will decrease the array length by 1  
    arr.pop();  
    // get the length of the array  
    uint length = arr.length;  
    // Delete does not change the array length.  
    // It resets the value at index to it's default value,  
    // in this case it resets the value at index 1 in arr2 to 0  
    uint index = 1;  
    delete arr2[index];  
}
```



```

// create array in memory, only fixed size can be created
uint[] memory a = new uint[](5);
// create string in memory
string memory hi = "hi";
}

```

```

}

```

Structs

The concept of structs exists in many high level programming languages. They are used to define your own data types which group together related data.

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.10;

contract TodoList {
    // Declare a struct which groups together two data types
    struct TodoItem {
        string text;
        bool completed;
    }

    // Create an array of TodoItem structs
    TodoItem[] public todos;

    function createTodo(string memory _text) public {
        // There are multiple ways to initialize structs

        // Method 1 - Call it like a function
        todos.push(TodoItem(_text, false));

        // Method 2 - Explicitly set its keys
        todos.push(TodoItem({ text: _text, completed: false }));
    }
}

```



```

        // Method 3 - Initialize an empty struct, then set individual
properties
        TodoItem memory todo;
        todo.text = _text;
        todo.completed = false;
        todos.push(todo);
    }

    // Update a struct value
    function update(uint _index, string memory _text) public {
        todos[_index].text = _text;
    }

    // Update completed
    function toggleCompleted(uint _index) public {
        todos[_index].completed = !todos[_index].completed;
    }
}

```

View and Pure Functions

This is a revision to what we have done in the previous tasks in solidity. You might have noticed that some of the functions we have been writing specify one of either a view or pure keyword in the function header. These are special keywords which indicate specific behavior for the function.

Getter functions (those which return values) can be declared either view or pure.

- View: Functions which do not change any state values
- Pure: Functions which do not change any state values and also do not read any state values

```
// SPDX-License-Identifier: MIT
```



```
pragma solidity ^0.8.10;
```

```
contract ViewAndPure {  
  
    // Declare a state variable  
    uint public x = 1;  
  
    // Promise not to modify the state (but can read state)  
    function addToX(uint y) public view returns (uint) {  
        return x + y;  
    }  
  
    // Promise not to modify or read from state  
    function add(uint i, uint j) public pure returns (uint) {  
        return i + j;  
    }  
}
```

▶ [Errors and warnings](#)

▶ [Memory storage and calldata](#)

Mappings

▶ [Mappings](#)

Mapping in Solidity is like a dictionary or lookup table. It connects keys to values, allowing you to store and retrieve data efficiently. Think of it as a

database where you can look up information using a specific identifier.

To create a mapping, use the syntax: `mapping(keyType => valueType) variableName`. For example, `mapping(address => uint) public balances;` creates a mapping that stores a number (`uint`) for each Ethereum address. You can then use `balances[userAddress] = 100;` to assign values and `balances[userAddress]` to retrieve them.

Unlike arrays, mappings don't track their entries or their size. They initialize all possible keys with default values (`0`, `false`, etc.), and you can't iterate through all entries. Mappings are useful for token balances, user data storage, or any situation where you need to associate data with unique identifiers.

In the previous tasks, we learned how to deploy smart contracts and understood the process involved. Building on that foundation and incorporating the new concepts introduced above, here's another tutorial that demonstrates how to deploy a smart contract using these newly covered topics.

▶ [Deploying a Smart Contract](#)